Memoria: Implementación del SIEM Wazuh en SOC Copilot

Oct 13, 2026 · @Vale

Resumen ejecutivo

SOC Copilot recibe ya en tiempo real las alertas de un SIEM Wazuh 4.12 desplegado sobre Ubuntu Server 22.04, y las muestra en su página SIEM con nivel, regla, agente, riesgo y técnica MITRE, listas para el análisis con IA. La prueba de punta a punta (una inyección SQL simulada contra la web) genera una alerta de nivel 10 mapeada a T1190 que aparece en la aplicación en segundos.

Los objetivos de la práctica eran:

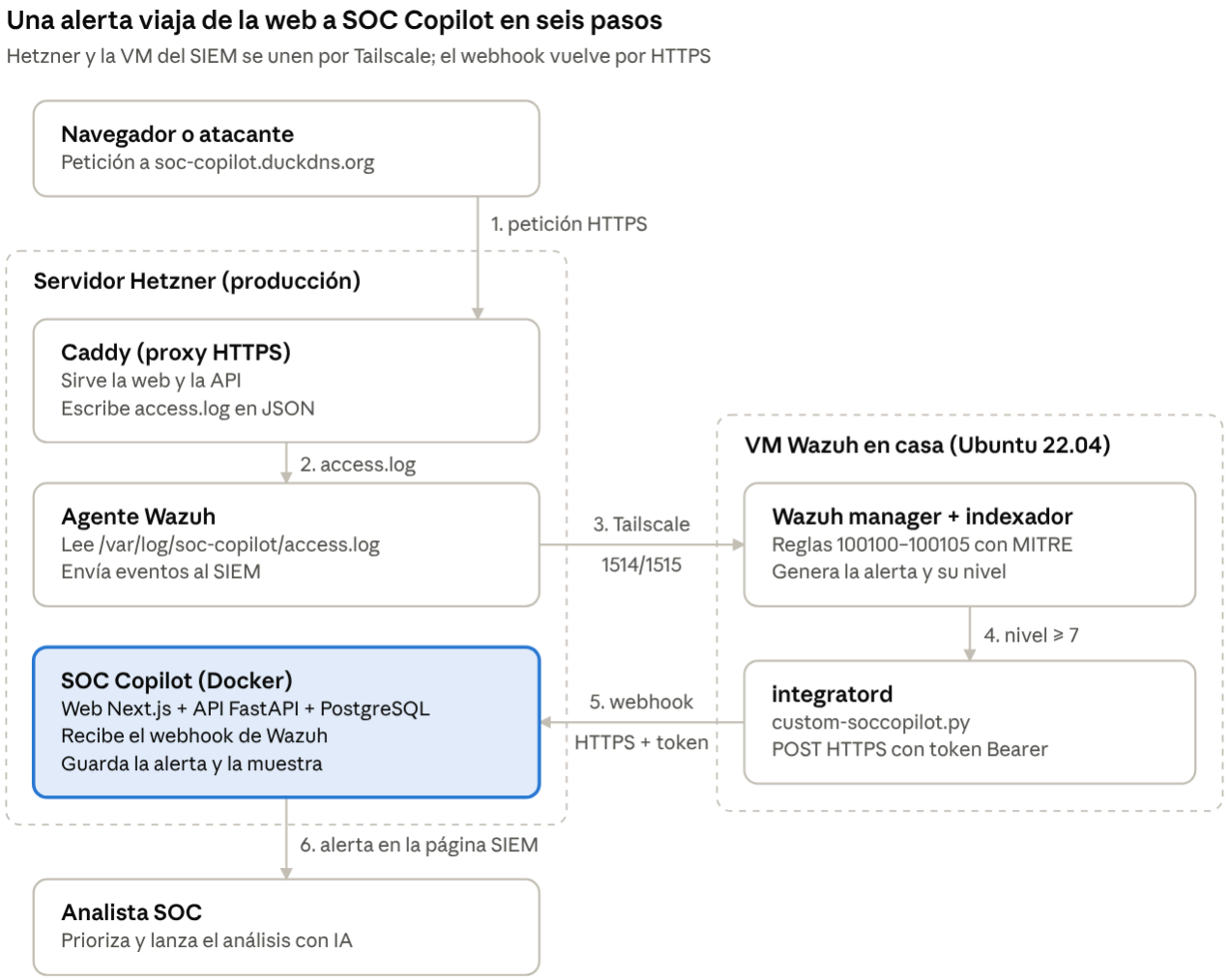
- Desplegar un SIEM funcional y documentar la elección de plataforma.
- Monitorizar el servidor de producción de SOC Copilot (Hetzner) y, en particular, el tráfico web de la aplicación.
- Detectar ataques web con reglas propias mapeadas a MITRE ATT&CK.
- Integrar las alertas en SOC Copilot para que un analista junior las priorice y las entienda con ayuda de la IA.
- Desplegar en producción la versión de la aplicación con la integración Wazuh, MFA obligatorio y recuperación de contraseña.

El trabajo incluyó también unificar dos versiones divergentes del código (la de producción y la de la Práctica 2) antes de poder desplegar.

Desde entonces, la parte del SIEM se completó con la gestión de sus alertas en SOC Copilot (asignación, escalado por niveles, revisión del Instructor y cierre masivo), con su operación diaria (ruido del monitor de integridad, parcheo del servidor a partir del detector de vulnerabilidades, disponibilidad de la VM) y con un simulador para probar la integración sin un Wazuh propio. El flujo de pruebas E2E, que incluye la integración con Wazuh, pasa en verde desde el 13 de octubre.

Arquitectura final

El sistema une dos máquinas: el servidor de producción en Hetzner, donde corre SOC Copilot, y una VM con Wazuh en la red de casa. Los eventos viajan al SIEM por Tailscale y las alertas vuelven a la aplicación por HTTPS.



arquitectura y flujo de una alerta · 6 pasos

El recorrido no necesita abrir ningún puerto del SIEM a internet: el agente entra por la red privada y el SIEM solo hace una conexión de salida hacia la web pública.

Elección del sistema operativo: Ubuntu Server 22.04 en lugar de Debian 13

Se eligió Ubuntu Server 22.04.5 LTS porque es un sistema soportado oficialmente por el instalador de Wazuh 4.12, mientras que Debian 13 (Trixie) no lo está y bloqueó la instalación. El primer intento se hizo sobre Debian 13 y se abandonó tras varios fallos encadenados.

Criterio	Debian 13 (Trixie)	Ubuntu Server 22.04 LTS
Soporte del instalador de Wazuh 4.12	No listado; el asistente avisa de que puede fallar	Listado como sistema recomendado

Criterio	Debian 13 (Trixie)	Ubuntu Server 22.04 LTS
Dependencia `software-properties-common`	Eliminada de la distribución; el instalador aborta	Disponible
Entorno gráfico	Instalado con escritorio (consume RAM)	Imagen *server* sin escritorio
Configuración inicial	Usuario sin `sudo`, repositorio apuntando al DVD, sin `curl`	OpenSSH incluido en el instalador, `curl` y `sudo` listos
Ciclo de soporte	Versión reciente, poco probada con Wazuh	LTS con soporte hasta 2027

El fallo decisivo en Debian 13 fue que el asistente de Wazuh intenta instalar `software-properties-common`, un paquete que Debian dejó de distribuir en esta versión. Se valoró crear un paquete vacío con `equivs` para engañar al instalador, pero se descartó: el sistema seguiría sin soporte oficial y otros pasos podrían fallar más adelante.

Se optó por la imagen *live-server* (sin escritorio) porque toda la RAM disponible queda para el indexador de Wazuh. La administración se hace por SSH desde Windows y el panel se consulta desde el navegador, por lo que no hace falta interfaz gráfica.

Despliegue del servidor Wazuh

El SIEM corre como instalación *all-in-one* de Wazuh 4.12 (indexador, servidor y panel en una sola máquina) dentro de una máquina virtual de VirtualBox con Ubuntu Server 22.04.

Máquina virtual

Recurso	Valor	Motivo
RAM	4 GB mínimo (8 GB recomendado)	El asistente aborta por debajo de 4 GB
CPU	2 núcleos	Mínimo exigido por el asistente
Disco	50 GB	El indexador crece rápido con las alertas
Red	Adaptador puente	Con NAT (10.0.2.15) no es accesible desde otros equipos
Acceso	OpenSSH Server activado en la instalación	Administración remota desde Windows

La VM obtuvo la IP local 192.168.1.209 en la red de casa.

Instalación

Se usó el asistente oficial con la opción `-a` (todos los componentes en un nodo):

```
curl -sO https://packages.wazuh.com/4.12/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

Al terminar, el asistente muestra la contraseña del usuario `admin` del panel y la guarda también en `wazuh-install-files.tar`. El panel queda accesible en `https://192.168.1.209`.

Ampliación del disco

El instalador de Ubuntu solo asignó 24 GB de los 48 GB del disco al volumen lógico raíz (comportamiento por defecto con LVM), y el disco llegó al 100 %. Se liberó espacio temporal y se amplió el volumen con el espacio libre del grupo:

```
sudo apt clean && sudo journalctl --vacuum-size=50M
sudo lvextend -l +100%FREE /dev/ubuntu-vg/ubuntu-lv
sudo resize2fs /dev/ubuntu-vg/ubuntu-lv
```

El resultado fue un volumen de 48 GB con un 49 % de uso.

Firewall (UFW)

Se activó UFW permitiendo solo lo necesario, siempre con SSH primero para no perder el acceso:

Puerto	Uso	Origen permitido
22/tcp	SSH de administración	Cualquiera
443/tcp	Panel de Wazuh	Cualquiera (restringible por IP)
1514/tcp	Eventos de los agentes	Interfaz de Tailscale y red local 192.168.1.0/24
1515/tcp	Registro de agentes	Interfaz de Tailscale y red local 192.168.1.0/24

```
sudo ufw allow 22/tcp
sudo ufw allow 443/tcp
sudo ufw allow in on tailscale0 to any port 1514,1515 proto tcp
sudo ufw enable
```

Conexión del servidor de Hetzner al SIEM

El servidor de producción `soc-copilot-hetzner` (Ubuntu 24.04, amd64) envía sus eventos al SIEM a través de una red privada Tailscale, porque está en internet y el SIEM en una red doméstica con IP privada.

Instalación del agente

El comando se generó desde el panel (*Agents management → Deploy new agent → Linux DEB amd64*) y se ejecutó en el servidor de Hetzner, no en el SIEM, que ya se monitoriza a sí mismo como agente `000`:

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.12.0-1_amd64.deb
sudo WAZUH_MANAGER='<IP del SIEM>' WAZUH_AGENT_NAME='soc-copilot-hetzner' dpkg -i ./wazuh-agent_4.12.0-1_amd64.deb
sudo systemctl daemon-reload && sudo systemctl enable --now wazuh-agent
```

Por qué Tailscale

Un servidor en internet no puede alcanzar la IP privada 192.168.1.209. Se compararon tres alternativas:

Opción	Ventaja	Inconveniente	Decisión
Tailscale (VPN WireGuard)	Sin abrir puertos del router; cifrado; gratis	Depende de un servicio externo	Elegida
Redirección de puertos en el router	Sin software extra	Expone puertos de casa a internet; IP pública cambiante	Descartada
Wazuh en un servidor en la nube	Disponible 24/7	Coste y nueva instalación	Trabajo futuro

Tailscale se instaló en la VM y en Hetzner con la misma cuenta. La VM recibió la IP 100.68.198.55, que es la que usa el agente como `<address>` en `/var/ossec/etc/ossec.conf`. La conectividad se comprobó con `tailscale ping` (45 ms) y `nc -zv` a los puertos 1514 y 1515.

Diagnóstico de la conexión

El agente tardó en registrarse por tres problemas que se resolvieron en orden:

1. **IP equivocada**: el agente apuntaba a 178.105.51.187, que resultaba ser la IP pública del propio servidor de Hetzner. Se detectó con `curl -4 ifconfig.me`.
2. **IP privada inalcanzable**: al corregirla a 192.168.1.209, el error `(1208) Unable to connect to enrollment service` persistió. Se resolvió con Tailscale.
3. **Disco del SIEM lleno**: el registro devolvía `Duplicate agent name`. El log del manager reveló la causa real, `No space left on device`: el SIEM entregaba una clave al agente pero no podía guardarla en `client.keys`. Tras ampliar el disco, se borraron los registros a medias con `manage_agents -r`, se vació `client.keys` en el agente y este se registró correctamente (`Connected to the server`).

Monitorización de la aplicación web

Wazuh vigila el tráfico web de SOC Copilot leyendo los logs de acceso de Caddy, el proxy inverso que sirve la aplicación, y los evalúa con seis reglas propias mapeadas a MITRE ATT&CK.

Logs de Caddy fuera del contenedor

La aplicación corre en Docker (contenedores `web`, `api`, `postgres`, `chroma` y `caddy`), y Caddy no guarda logs de acceso por defecto. Se añadieron dos cambios:

- En `infra/caddy/Caddyfile`, dentro del bloque del dominio: `log { output file /var/log/caddy/access.log; format json }`.
- En `infra/docker-compose.prod.yml`, un volumen en el servicio `caddy`: `/var/log/soc-copilot:/var/log/caddy`.

Antes de aplicar se validó el Caddyfile (`caddy validate`) y el compose (`docker compose config`), y se recreó solo Caddy con `--no-deps` y el `.env` de producción, para no tocar la API ni la web. Caddy rota el archivo de log automáticamente.

Lectura en el agente

En el `ossec.conf` del agente de Hetzner:

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/soc-copilot/access.log</location>
</localfile>
```

Reglas personalizadas

Wazuh no trae reglas para Caddy, así que se escribieron en `/var/ossec/etc/rules/local_rules.xml` del SIEM:

ID	Nivel	Detecta	MITRE
100100	0	Cualquier petición de Caddy (regla base)	—
100101	5	Respuesta con error 4xx	—
100102	6	Escaneo de rutas sensibles (`wp-login` , `.env` , `.git/` , `phpmyadmin` ...)	T1595 Active Scanning
100103	10	Path traversal (`../` , `%2e%2e` , `/etc/passwd`)	T1190 Exploit Public-Facing Application
100104	10	Patrones de SQLi o XSS (`union select` , `or 1=1` , `<script` ...)	T1190 Exploit Public-Facing Application
100105	10	20 errores 4xx en 60 s desde la misma IP	T1595 Active Scanning

Dos detalles del formato de reglas obligaron a corregir la primera versión:

- El carácter `<` dentro de una expresión debe escribirse `<` porque el archivo es XML.
- `status` es un campo reservado de Wazuh y no admite `<field name="status">` ; se usa su etiqueta propia, `<status>^4</status>` .

Las reglas se validaron con `wazuh-analysisd -t` y con `wazuh-logtest` , pegando una línea JSON de ejemplo: la regla 100104 saltó con nivel 10, táctica *Initial Access* y técnica T1190.

Integración Wazuh → SOC Copilot

Wazuh envía cada alerta de nivel 7 o superior a SOC Copilot mediante un webhook HTTPS autenticado con un token *Bearer* (modo *push*). La API de SOC Copilot ya ofrecía dos modos, y se eligió este:

Modo	Cómo funciona	Por qué sí o no
Push (webhook)	`integratord` de Wazuh hace `POST /api/integrations/wazuh/webhook` con cada alerta	Elegido: el SIEM solo necesita salida a internet, sin exponer ningún puerto

Modo	Cómo funciona	Por qué sí o no
Pull (indexador)	La API consulta periódicamente el Wazuh Indexer (puerto 9200)	Descartado: el indexador solo escucha dentro de la VM y habría que exponerlo

Lado de SOC Copilot

La integración se activa con variables del `.env` de producción. Con el token vacío el endpoint responde 503 (desactivado); con token, 401 si falta o no coincide.

Variable	Valor	Función
`WAZUH_WEBHOOK_TOKEN`	64 caracteres hexadecimales (`openssl rand -hex 32`)	Secreto compartido con el SIEM
`WAZUH_MIN_RULE_LEVEL`	7	Por debajo, las alertas se aceptan pero no se guardan
`WAZUH_INDEXER_URL`	vacío	Desactiva el modo pull
`WAZUH_POLL_INTERVAL_SECONDS`	0	Sin sondeo periódico

Lado del SIEM

Se usó el script del propio repositorio, `integrations/wazuh/custom-soccopilot.py` . Solo usa la biblioteca estándar de Python, reintenta 3 veces ante errores de red o 5xx y registra los fallos en `/var/ossec/logs/integrations.log` .

```
sudo cp custom-soccopilot.py /var/ossec/integrations/
sudo chown root:wazuh /var/ossec/integrations/custom-soccopilot.py
sudo chmod 750 /var/ossec/integrations/custom-soccopilot.py
```

En `/var/ossec/etc/ossec.conf` se añadió un bloque `<integration>` con el nombre `custom-soccopilot.py` , la URL del webhook, el token como `<api_key>` , `<level>7</level>` y formato JSON. El token se introdujo con `read -rsp` para que no apareciera en pantalla.

Dos ajustes necesarios

1. **Certificados**: el script se ejecuta con el Python que incluye Wazuh, que no conoce los certificados de confianza del sistema, y fallaba con `CERTIFICATE_VERIFY_FAILED`. Se cambió el valor por defecto de `CA_FILE` a `/etc/ssl/certs/ca-certificates.crt`.
2. **Token vacío**: tras corregir lo anterior, la API respondía 401. Comparando la huella SHA-256 del token en ambos lados, la del SIEM correspondía a una cadena vacía: la variable no se había leído al añadir el bloque. Se rellenó con `sed`, verificando antes que el token leído tenía 64 caracteres y que las huellas coincidían.

Unificación del código y despliegue en producción

El webhook de Wazuh no existía en producción (respondía 404) porque el código de la Práctica 2 estaba en otro repositorio de GitHub y había divergido del de producción. Se unieron ambas versiones con git, se probaron en local y se desplegaron como la Pull Request #10.

Diagnóstico de la divergencia

- Las migraciones de base de datos compartían `0001` a `0004` y luego se separaban: producción tenía `0005_group_messages` (chat de grupo) y la Práctica 2 tenía `0005_wazuh_integration`, `0006_mfa_totp` y `0007_password_reset`.
- Producción incluía funciones ausentes en la Práctica 2: chat de grupo, botón flotante, selector de idioma y tema en el login y barra lateral plegable.
- Desplegar la Práctica 2 tal cual habría borrado esas funciones y la API no habría arrancado, porque la base de datos estaba en una migración que su código no conocía.

Estrategia de unión

1. **Encontrar el punto de partida común**: se copió el código de la Práctica 2 en una rama temporal y se comparó con varios commits de producción (`git diff --shortstat`). El de menos diferencias, `969f0df` (92 archivos), era la base.
2. **Rama sobre la base**: rama `practica2` desde `969f0df` con el código de la Práctica 2, y merge con `main` en una rama aparte. Git unió solo la mayoría de archivos y dejó 21 conflictos.
3. **Resolución**: se tomó la versión de la Práctica 2 en los conflictos y se reincorporaron a mano las piezas de producción: enlace al chat de grupo en el menú, sus textos en español, inglés y francés, y la página `/logs`, que se había renombrado por error.
4. **Cadena de migraciones**: `0005_wazuh_integration` pasó a depender de `0005_group_messages`, quedando `0004 → 0005_group_messages → 0005_wazuh_integration → 0006_mfa_totp → 0007_password_reset`.

5. **Errores de compilación**: `npx tsc --noEmit` detectó 6 errores porque producción había traducido las etiquetas de fortaleza de contraseña (`STRENGTH_LABEL_KEYS`, `labelKey`). Se adaptaron tres pantallas y se añadieron 11 textos por idioma.
6. **CI**: el lint de Python (`ruff`) falló por 2 imports desordenados (I001), corregidos con `ruff check --fix`. Después, lint, migraciones y tests de la API, web e infraestructura pasaron en verde.

Prueba en local

Se levantó todo con Docker Compose en el PC (web en el puerto 13500, API en 8080). Se comprobó que las migraciones llegaban a `0007_password_reset`, el registro con MFA y todas las funciones de ambos lados. Hubo que resolver varios problemas de entorno, descritos en *Incidencias*.

Despliegue en producción

1. Copia de seguridad: `pg_dump -Fc` de la base de datos (33 KB).
2. Código: `git stash`, `git pull` y `git stash pop` para conservar el bloque de logs del Caddyfile.
3. `.env`: se comprobó que existía `APP_ENCRYPTION_KEY` (la necesita el MFA) y se añadieron las variables de MFA, recuperación de contraseña y Wazuh.
4. Despliegue con el script del proyecto: `sudo -u soc bash /opt/soc-copilot/scripts/deploy.sh`, que valida el `.env`, reconstruye y arranca.
5. Verificación: `alembic current` devolvió `0007_password_reset (head)`, `/api/health` respondió 200 y el webhook sin token respondió 401 *invalid webhook token*.

Con `MFA_REQUIRED=true` todos los usuarios configuran el doble factor al entrar; la API se niega a arrancar en producción si se desactiva.

Medidas de seguridad aplicadas

La integración se diseñó para no exponer servicios del SIEM a internet y para que ningún secreto viaje sin cifrar ni quede en el repositorio.

Área	Medida
Red del SIEM	Puertos de agentes (1514/1515) solo por Tailscale y red local; el indexador (9200) nunca se expone
Transporte	Agente → SIEM cifrado por Tailscale (WireGuard); SIEM → SOC Copilot por HTTPS con verificación de certificado

Área	Medida
Autenticación del webhook	Token aleatorio de 256 bits; sin token o con token incorrecto la API responde 401
Gestión de secretos	Tokens y claves solo en `.env` (permisos 600, fuera de git); el token se introdujo con `read -rsp` y se comparó por hash SHA-256, sin mostrarlo
Cuentas de usuario	MFA/TOTP obligatorio, secretos TOTP cifrados con `APP_ENCRYPTION_KEY` (Fernet); niveles altos (L2, Instructor) requieren aprobación de un administrador
Ruido de alertas	Solo se envían alertas de nivel 7 o superior
Despliegue	Copia de seguridad previa de la base de datos; cambios validados en local y por CI antes de producción; repositorio gestionado con el usuario de servicio `soc`
Firewall	UFW activo en el SIEM con política de mínimo privilegio

Pruebas y validación

La prueba final confirmó el circuito completo: una petición con un patrón de inyección SQL contra la web de producción aparece en la página SIEM de SOC Copilot como alerta de nivel 10, riesgo HIGH y técnica T1190.

Prueba	Cómo	Resultado esperado	Resultado
Reglas de Caddy	`wazuh-logtest` con una línea JSON de SQLi	Regla 100104, nivel 10, T1190	Correcto
Agente de Hetzner	Log del agente	`Connected to the server`	Correcto
Endpoint del webhook sin token	`curl -X POST`	401 *invalid webhook token*	Correcto
Webhook con token	`curl` con alerta de ejemplo y cabecera *Bearer*	2xx y alerta visible en SOC Copilot	Correcto

Prueba	Cómo	Resultado esperado	Resultado
Migraciones en producción	`alembic current`	`0007_password_reset (head)`	Correcto
CI de la Pull Request	GitHub Actions	API, web e infraestructura en verde	Correcto
Punta a punta	Visitar `/?id=1%20union%20select` en la web	Alerta en la página SIEM en menos de 1 minuto	Correcto

En la prueba de punta a punta, cada visita produjo una alerta «Caddy: posible SQLi/XSS desde 79.117.252.84» del agente `soc-copilot-hetzner`, con estado *Pendiente* y el botón de análisis con IA disponible.

Mejoras del 9 de octubre: navegación SIEM, análisis desde Wazuh y Gemini 3.x

El 9 de octubre se agrupó la navegación bajo un menú SIEM, el botón «Analizar con IA» pasó a abrir la alerta en la página de análisis y la IA se migró a Gemini 3.x, porque los modelos 2.5 ya no están disponibles para claves nuevas. Los cambios se integraron con la rama `siem-menu-y-modelo` y el CI en verde.

Cambios en la aplicación

Cambio	Archivos	Detalle
Menú SIEM desplegable	`AppShell.tsx`, `i18n.tsx`	Agrupa Alertas Wazuh (`/siem`), Alertas (`/alerts`) y Logs (`/logs`); se abre solo si la página activa es una de ellas; las direcciones no cambian
«Analizar con IA» abre la página Alertas	`siem/page.tsx`, `alerts/page.tsx`	El botón lleva a `/alerts?wazuh_id=<id>`, que muestra nivel, agente y fecha, carga el log en solo lectura y lanza el análisis si estaba pendiente; si ya estaba analizada muestra el análisis guardado; reanalizar no crea duplicados

Cambio	Archivos	Detalle
Modelos Gemini 3.x	`config.py`, `.env.example`	Predeterminado `gemini-3.5-flash-lite`; lista permitida: `gemini-3.5-flash-lite`, `gemini-3.5-flash`, `gemini-3.1-flash-lite`
Mostrar u ocultar la contraseña	`PasswordInput.tsx`, `login/page.tsx`, `i18n.tsx`	Componente nuevo con sus textos en español, inglés y francés
Certificados en la integración	`custom-soccopilot.py`	`CA_FILE` apunta por defecto a `/etc/ssl/certs/ca-certificates.crt`, ya incluido en el repositorio

Diagnóstico del error 502 de la IA

1. El análisis devolvía `502 AI provider error` aunque la clave de Gemini estaba configurada.
2. Una llamada directa a Gemini desde el contenedor de la API, con la misma clave, dio `404` para `gemini-2.5-flash-lite` («no longer available to new users») y `200` para `gemini-3.1-flash-lite` y `gemini-3.5-flash`.
3. El nombre de modelo de la barra superior es un texto fijo; el modelo real lo decidían el selector de la página y la lista permitida de `config.py`, que solo contenía modelos 2.x.
4. Tras cambiar la lista, la web seguía enviando `gemini-2.5-flash` (visto en F12 → Network → Payload). Venía de la preferencia guardada del usuario en Settings (columna `preferred_chat_model`), no del navegador; al cambiarla, el análisis funcionó.

Pruebas

- Dos logs sintéticos para la página Logs, con IPs de rangos reservados para documentación: `prueba-auth.log` (fuerza bruta SSH con acceso conseguido, persistencia y borrado de logs) y `prueba-web-access.log` (Nikto, sqlmap, path traversal, XSS, credential stuffing y webshell).
- Una alerta de Wazuh simulada enviada al webhook local con `Invoke-WebRequest`, para probar el flujo sin el SIEM.
- CI: un test de la API esperaba 200 y recibía 422 porque guardaba `gemini-2.5-flash-lite` como modelo preferido. Se actualizaron los modelos en `test_byo_llm.py` y `test_smoke.py`, manteniendo el test que comprueba el rechazo de un modelo no permitido; los 67 tests de esos archivos pasaron.

- CI: el build de la web fallaba porque `PasswordInput.tsx` usaba los textos `pw_show` y `pw_hide`, que no existían; se añadieron en los tres idiomas. Se excluyó del commit `tsconfig.tsbuildinfo`, que es un artefacto de compilación.

Procedimiento de despliegue

1. Copia de seguridad de la base de datos con `pg_dump -Fc`.
2. En el `.env` de producción, `GEMINI_CHAT_MODEL=gemini-3.5-flash-lite` y, si existe, `GEMINI_CHAT_MODELS_ALLOWLIST` con la lista nueva.
3. `git stash`, `git pull` y `git stash pop` (para conservar el bloque de logs del Caddyfile) y `deploy.sh`.
4. Justo después, borrar las preferencias de modelo retiradas, para que esos usuarios usen el modelo por defecto:

```
UPDATE users SET preferred_chat_model = NULL
WHERE preferred_chat_model NOT IN ('gemini-3.5-flash-lite', 'gemini-3.5-flash', 'gemini-3.1-flash-lite');
```

La limpieza va después del despliegue porque, con la lista nueva activa, la API ya no permite volver a guardar un modelo retirado.

Gestión de las alertas del SIEM en SOC Copilot

Cada alerta de Wazuh tiene un ciclo de vida y lo que cada analista puede hacer con ella depende de su nivel SOC. Las reglas se comprueban en la API (`services/alert_workflow.py`); el detalle completo está en `docs/20-ciclo-vida-alertas.md` del repositorio.

Estado	Significado
Nueva	Ha llegado de Wazuh y nadie la ha cogido
Asignada	Un analista la trabaja («Asignada a: nombre»); solo entonces puede analizarla con IA
Escalada	Un L1 la pasa a L2 con una nota obligatoria
Resuelta / Falso positivo	Cerrada con su resolución

- Un **L1** solo se asigna alertas nuevas y solo cierra las de riesgo bajo o medio; las altas, críticas o sin analizar las escala.
- Un **L2** se asigna las escaladas, las investiga, las cierra y puede reabrirlas.

- El **Instructor** valora cada decisión como Correcta, Mejorable o Incorrecta desde la cola **Para revisar**.
- La tabla de **SIEM · Alertas Wazuh** se organiza en colas (Nuevas, Mis alertas, Asignadas, Escaladas, Cerradas, Para revisar) y permite el **cierre masivo**: varias alertas con una única resolución, aplicando las mismas reglas alerta por alerta.
- Cada acción queda en el historial de la alerta (`alert_events`) y en la auditoría. Migraciones `0009` a `0011`.

Operación del SIEM

Ruido del monitor de integridad

Las alertas «Integrity checksum changed» no venían de los despliegues de SOC Copilot, sino de las actualizaciones del sistema. En el histórico guardado (7, 8, 9 y 12 de octubre) había 87, todas del agente `soc-copilot-hetzner`:

Archivos	Origen
Decenas de `/usr/bin/sg_*`	Actualización del paquete `sg3-utils`
`/boot/vmlinuz` , `/boot/initrd.img` (y sus `.old`), `/boot/grub/grub.cfg`	Actualización del kernel
`/etc/ld.so.cache`	Caché de librerías regenerada al instalar paquetes

Se decidió **no silenciarlas**: son los archivos que modificaría un atacante para persistir. Se concentran las actualizaciones en una ventana conocida, se comprueban contra `/var/log/apt/history.log` y se cierran juntas con el cierre masivo. Para el recuento, los comandos con comodines se ejecutan dentro de `sudo bash -c '…'` , porque el usuario normal no puede leer las carpetas de alertas de Wazuh y el comodín no se expandía.

Detector de vulnerabilidades y parcheo

El detector de vulnerabilidades de Wazuh avisó de decenas de CVE en el kernel `linux-image-6.8.0-146-generic` del servidor de producción. Se aplicó el procedimiento de parcheo: copia de seguridad de la base de datos, `apt full-upgrade` , reinicio, comprobación de `uname -r` , de los contenedores, del agente de Wazuh y de Tailscale, y `apt autoremove --purge` para retirar el kernel antiguo. Las CVE se marcan como resueltas en el siguiente escaneo; las que no tengan parche publicado se consultan con `pro fix CVE-…` .

Disponibilidad y acceso a la VM

- El histórico no tiene alertas de los días 10 y 11 de octubre: la VM estuvo apagada y no se registró nada.
- Tras un reinicio, la VM pasó de `192.168.1.209` a `192.168.1.208` por DHCP y el panel dejó de estar en la dirección habitual. La conexión con producción no se vio afectada porque usa la IP de Tailscale (`100.68.198.55`), que también sirve para entrar por SSH al SIEM desde el servidor de Hetzner.

Probar sin Wazuh: simulador

El repositorio incluye `integrations/wazuh/wazuh_simulator.py` , que envía alertas con el mismo formato y por el mismo webhook que Wazuh. Se añadió el volumen `../integrations:/code/integrations:ro` al contenedor de la API del entorno local, porque antes el comando del README fallaba con «No such file». Resultados de la prueba:

Prueba	Resultado
`--count 8 --interval 1`	Las alertas de nivel ⩾ 7 responden `created: 1` y aparecen en la cola Nuevas
Alertas de nivel inferior a 7	`below_threshold: 1` (descartadas a propósito)
Dos veces con `--seed 42 --count 5`	La segunda vez, `duplicates: 1` (deduplicación)

El test `test_e2e_wazuh.py` del flujo E2E recorre la integración contra la API; se adaptó para asignarse la alerta antes de analizarla.

Incidencias y lecciones aprendidas

La mayoría de los bloqueos vinieron del entorno (sistema operativo, red, disco, rutas de Windows) y no del propio Wazuh; leer el log del componente que falla fue siempre más rápido que probar soluciones a ciegas.

Problema	Causa	Solución
`ssh-keygen` no reconocido en Windows	PowerShell x86 (32 bits) no ve `System32\OpenSSH`	Usar la PowerShell de 64 bits
Usuario sin `sudo` en Debian	Contraseña de root fijada al instalar	`usermod -aG sudo` como root
`apt` pedía el DVD	Repositorio `cdrom:` activo	Comentarlo y añadir los repositorios en línea

Problema	Causa	Solución
Instalador de Wazuh abortaba	Debian 13 sin `software-properties-common` y VM con menos de 4 GB	Cambiar a Ubuntu 22.04 con 4 GB y 2 CPU
Disco del SIEM al 100 %	LVM solo usaba 24 de 48 GB	`lvextend` + `resize2fs`
Agente sin conectar	IP del propio servidor y luego IP privada	IP de Tailscale
`Duplicate agent name`	El SIEM no podía guardar `client.keys` (disco lleno)	Ampliar disco, borrar registros a medias y reinscribir
Reglas sin cargar	`<` sin escapar y campo reservado `status`	`<` y `<status>`
Webhook 404	Producción con código anterior al webhook	Unificar versiones y desplegar
`npm install` fallaba	Ruta con `&` y dentro de OneDrive	Mover el repositorio a `C:\dev`
Puerto 55432 ocupado	Otra copia del proyecto levantada en Docker	`docker compose -p soc-copilot-p2 down`
API sin base de datos (`Name or service not known`)	Red de Docker inconsistente tras un arranque fallido	`docker compose down` y `up`
`CERTIFICATE_VERIFY_FAILED`	Python de Wazuh sin certificados del sistema	`CA_FILE` apuntando a `ca-certificates.crt`
Webhook 401	Token vacío en `ossec.conf`	Rellenarlo y comparar huellas SHA-256

Incidencias posteriores, ya con el SIEM en marcha:

Problema	Causa	Solución
Panel del SIEM inaccesible	La VM cambió de IP por DHCP al reiniciar	Nueva IP local; fijarla queda pendiente. Producción sigue por Tailscale
Sin alertas los días 10 y 11	La VM estuvo apagada	Limitación de tener el SIEM en un equipo local

Problema	Causa	Solución
Ráfagas de «Integrity checksum changed»	Actualizaciones de paquetes y del kernel	No se silencian: se comprueban y se cierran con el cierre masivo
`zgrep` sin resultados al contar alertas	El comodín lo expandía el usuario sin permisos, antes de `sudo`	Ejecutar el comando dentro de `sudo bash -c '…'`
Simulador: «No such file»	La carpeta `integrations/` no estaba en el contenedor de la API	Volumen `../integrations:/code/integrations:ro`
`test_e2e_wazuh.py` con 403	Analizaba una alerta sin asignársela	El test se asigna la alerta primero

Lecciones principales:

- Comprobar la matriz de sistemas soportados antes de elegir el sistema operativo.
- Un error puede ocultar otro: `Duplicate agent name` era una consecuencia del disco lleno.
- Trabajar en ramas y en una carpeta aparte antes de tocar producción, con copia de seguridad previa.
- Comparar secretos por su hash para depurar sin exponerlos.

Limitaciones y trabajo futuro

La limitación principal es que el SIEM corre en una VM de un PC personal: solo recibe y reenvía alertas mientras ese PC está encendido.

- [ ] Migrar Wazuh a un servidor en la nube (mínimo 4 GB de RAM y 2 vCPU, mejor 8 GB) para disponibilidad 24/7.
- [ ] Fijar la IP de la VM del SIEM (reserva DHCP en el router o IP fija con netplan).
- [x] ~~Subir al repositorio el ajuste de `CA_FILE` en `custom-soccopilot.py` para que la integración funcione a la primera en nuevas instalaciones.~~
- [ ] Rotar el token del webhook si aparece en capturas compartidas fuera del equipo.
- [x] ~~Revisar las vulnerabilidades de dependencias de la web (`npm audit`): 0 tras la corrección.~~

- [ ] Actualizar `passlib` / `bcrypt` para eliminar el aviso de versión de `bcrypt` en la API.
- [x] ~~Desactivar las cuentas de administrador cuya contraseña nadie conoce.~~
- [ ] Añadir endpoints Windows con Sysmon para detecciones de procesos, PowerShell y red.
- [ ] Borrar la copia de pruebas `/opt/soc-copilot-practica3` del servidor de producción.
- [x] ~~Actualizar la documentación del proyecto (README, `docs/` y `security.md`).~~
- [x] ~~Corregir la página Logs: al enviar un archivo a Alertas solo pasa la primera línea.~~
- [x] ~~Analizar el ruido de «Integrity checksum changed»: viene de actualizaciones del sistema y se gestiona con el cierre masivo, sin silenciarlo.~~
- [ ] Programar las actualizaciones automáticas del servidor en una ventana fija.
- [x] ~~Diferenciar las tareas por nivel SOC (L1, L2, Instructor): ciclo de vida con asignación, escalado, colas por nivel, revisión del Instructor y cierre masivo.~~
- [ ] Instructor: panel de equipo con métricas por analista y escenarios de entrenamiento con `wazuh_simulator.py`.

Anexo: configuraciones y comandos clave

Reglas personalizadas (`local_rules.xml`)

```
<group name="web,caddy,">
  <rule id="100100" level="0">
    <decoded_as>json</decoded_as>
    <field name="logger">^http.log.access</field>
    <description>Caddy: petición web</description>
  </rule>
  <rule id="100101" level="5">
    <if_sid>100100</if_sid>
    <status>^4</status>
    <description>Caddy: error 4xx desde $(request.remote_ip)</description>
  </rule>
  <rule id="100102" level="6">
    <if_sid>100100</if_sid>
    <field name="request.uri" type="pcre2">(?i)(wp-login|wp-
admin|phpmyadmin|\.env|\.git/|xmlrpc\.php)</field>
    <description>Caddy: escaneo de rutas sensibles desde $(request.remote_ip)
</description>
    <mitre><id>T1595</id></mitre>
  </rule>
  <rule id="100103" level="10">
    <if_sid>100100</if_sid>
    <field name="request.uri" type="pcre2">(?i)(\.\./|%2e%2e|/etc/passwd)
```

```
</field>
    <description>Caddy: intento de path traversal desde $(request.remote_ip)
</description>
    <mitre><id>T1190</id></mitre>
  </rule>
  <rule id="100104" level="10">
    <if_sid>100100</if_sid>
    <field name="request.uri" type="pcre2">(?i)
(union(\s|%20|\+)+select|or(\s|%20|\+)+1=1|sleep\(|&lt;script|%3cscript)
</field>
    <description>Caddy: posible SQLi/XSS desde $(request.remote_ip)
</description>
    <mitre><id>T1190</id></mitre>
  </rule>
  <rule id="100105" level="10" frequency="20" timeframe="60">
    <if_matched_sid>100101</if_matched_sid>
    <same_field>request.remote_ip</same_field>
    <description>Caddy: muchos errores 4xx desde $(request.remote_ip),
posible escaneo</description>
    <mitre><id>T1595</id></mitre>
  </rule>
</group>
```

Bloque de integración en el SIEM (`ossec.conf`)

```
<ossec_config>
  <integration>
    <name>custom-soccopilot.py</name>
    <hook_url>https://soc-
copilot.duckdns.org/api/integrations/wazuh/webhook</hook_url>
    <api_key>WAZUH_WEBHOOK_TOKEN</api_key>
    <level>7</level>
    <alert_format>json</alert_format>
  </integration>
</ossec_config>
```

Comandos de comprobación

Para qué	Dónde	Comando
Listar agentes	SIEM	`sudo /var/ossec/bin/agent_control -l`
Validar reglas	SIEM	`sudo /var/ossec/bin/wazuh-analysisd -t`
Probar una línea de log	SIEM	`sudo /var/ossec/bin/wazuh-logtest`

Para qué	Dónde	Comando
Ver fallos del webhook	SIEM	`sudo tail -f /var/ossec/logs/integrations.log`
Estado del agente	Hetzner	`sudo tail -n 20 /var/ossec/logs/ossec.log`
Versión de la base de datos	Hetzner	`sudo docker exec soc-copilot-prod-api-1 alembic current`
Desplegar la aplicación	Hetzner	`sudo -u soc bash /opt/soc-copilot/scripts/deploy.sh`
Copia de seguridad	Hetzner	`sudo docker exec soc-copilot-prod-postgres-1 sh -c 'pg_dump -U "$POSTGRES_USER" -d "$POSTGRES_DB" -Fc' > backup.dump`
Comparar el token sin mostrarlo	Ambos	`... | tr -d '\r\n' | sha256sum`